

HashMap - Complete Revision Notes (Version 2)

Made for quick interview revision in the same simple language used in our lessons. This version adds more diagrams, clearer flows and stronger visual revision points.

1. Why was HashMap created?

HashMap was created to find data quickly. If you store many elements in a normal list and search one by one, the search becomes slow. HashMap uses the key to jump to a bucket first, so it reduces search work a lot.

Big picture
ArrayList style search = check one by one
HashMap style search = jump to bucket first
Goal = fast lookup by key

2. What is a HashMap?

HashMap stores data as key-value pairs. The key is used to find the value.

Simple example:

```
map.put(101, "Rahul")
map.put(102, "Amit")
```

```
map.get(101) -> Rahul
map.get(102) -> Amit
```

Remember
Key = what you search with
Value = what you get back
HashMap has no index like ArrayList
HashMap does not guarantee order

3. Internal structure of HashMap

Internally HashMap uses an array called `table[]`. Every position of that array is called a bucket. A bucket can contain one node or many nodes if collisions happen.

```
table[]
+---+---+---+---+---+---+
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
+---+---+---+---+---+---+
          |
          v
        bucket 3
```

Bucket is just another way of saying array index. That is one of the most important ideas in HashMap.

4. The complete put() flow

When you call `put(key, value)`, Java does not scan the whole map. It first calculates `hashCode()`, then turns that hash into a bucket index, then checks that bucket.

```
put(key, value)
```

```
Key
|
v
hashCode()
|
v
bucket index
|
v
go to bucket
|
+--> bucket empty? ---> add new node
|
+--> bucket has nodes? -> compare with equals()
                        |
                        +--> same key? replace value
                        |
                        +--> different key? add another node
```

put() in simple words
Find the bucket first
Check whether key already exists
Replace value if key is same
Add new node if key is new

5. The complete get() flow

get() follows the same logic. Java takes the key, calculates the hash, goes to the bucket, and then compares keys in that bucket using equals() until it finds the exact match.

```
get(key)

Key
|
v
hashCode()
|
v
bucket index
|
v
go to bucket
|
v
check nodes one by one
|
v
equals() matches key
|
v
return value
```

If Java does not find the key, get() returns null.

6. hashCode() and equals()

This is the heart of HashMap. hashCode() decides which bucket to go to. equals() decides which key inside that bucket is the exact match.

Short rule:

```
hashCode() -> bucket
equals()    -> exact key
```

Why both are needed
hashCode() narrows the search to one bucket
equals() confirms the exact key inside that bucket
Both work together

7. What is collision?

Collision happens when two different keys land in the same bucket. This is normal. HashMap is built to handle collisions.

Collision example:

```
Rahul -> bucket 5
Amit -> bucket 5
```

Same bucket, different keys = collision

HashMap stores multiple nodes in the same bucket. Earlier Java versions used only a linked list in the bucket. Java 8+ can convert a crowded bucket into a tree.

Collision handling
Same bucket can store multiple nodes
equals() finds the exact key
Heavy collisions can become a tree

8. Why custom objects need special care

If you use your own class as a key, for example Student or Employee, Java must know when two objects should be treated as equal. That is why custom keys usually need both equals() and hashCode() overridden.

Example:

```
class Student {
    int id;
    String name;

    // override equals()
    // override hashCode()
}
```

If you override only equals(), logically equal objects may still go to different buckets. If you override only hashCode(), Java may go to the same bucket but equals() may still fail to confirm the key.

Golden rule for custom keys
Override equals()
Override hashCode()
Keep both consistent

9. The hashCode / equals contract

If two objects are equal according to equals(), then they must have the same hashCode(). This rule is important because HashMap uses hashCode() first and equals() after that.

Rule:

```
if a.equals(b) == true
then a.hashCode() == b.hashCode()
```

If this rule breaks, equal objects may go to different buckets and HashMap may not behave correctly.

10. Capacity, load factor and threshold

Capacity means the number of slots in the internal array. Default capacity is 16. Load factor tells how full the HashMap can get before resizing. Default load factor is 0.75. Threshold = capacity x load factor.

Default values:

```
capacity = 16
load factor = 0.75
threshold = 16 x 0.75 = 12
```

That means when the 13th element is inserted, HashMap becomes crowded enough to resize.

Why not wait until the map is full?
Waiting until 100 percent full creates too many collisions
Java resizes earlier to keep performance good

11. Resize and rehash

When size crosses the threshold, HashMap creates a new bigger array and moves the old entries into it. This is called resizing and rehashing.

Example:

```
capacity = 16
threshold = 12

13th insert
-> resize
-> capacity becomes 32
-> nodes are rechecked and moved
```

After resize, some nodes may stay in the same bucket and some may move to a different bucket. That is why rehashing matters.

Before resize:	After resize:
bucket 5	bucket 5
v	v
Rahul	Rahul
bucket 5	bucket 21
v	v
Amit	Amit

12. Treeification in Java 8+

If one bucket becomes too crowded, Java 8 can convert that bucket from a linked list into a Red-Black Tree. This helps the worst-case search inside that bucket.

Treeification happens when:

```
bucket size >= 8
AND
table capacity >= 64
```

If the table is still small, Java prefers resizing first because resizing may naturally spread the nodes into different buckets.

Why 64?
If the table is too small, resizing may solve the collision problem
Treeification is used when the table is already large enough and collisions are still heavy

13. Null key and null values

HashMap allows one null key and multiple null values. The null key is treated specially and goes into one fixed bucket internally.

Examples:

```
map.put(null, "value");    // allowed once
map.put("A", null);       // allowed
map.put("B", null);       // allowed
```

This is a small but important interview detail. Many people remember that HashMap allows nulls, but forget that only one null key is allowed.

14. Time complexity

Operation	Average	Reason
put()	O(1)	Direct bucket lookup
get()	O(1)	Direct bucket lookup
remove()	O(1)	Direct bucket lookup
Worst case	O(n)	Many collisions in one bucket
Treeified bucket	O(log n)	Red-Black Tree search

Average performance is fast because HashMap tries to keep buckets small. Worst case happens when many keys collide into one bucket.

15. HashMap vs HashSet vs TreeMap

Collection	Main idea	Important point
HashMap	Key-value storage	Fast lookup by key
HashSet	Unique elements	Internally uses HashMap
TreeMap	Sorted key-value storage	Uses Red-Black Tree

Use HashMap when you need speed. Use HashSet when you need uniqueness. Use TreeMap when you need sorted keys.

16. Common mistakes

- 1) Thinking bucket means something other than array index.
- 2) Overriding only equals() or only hashCode().
- 3) Expecting HashMap to preserve order.

- 4) Thinking resize simply copies data without rehashing.
5) Assuming hashCode() alone can identify the exact key.

Very short memory line
hashCode() -> bucket
equals() -> exact key
collision -> same bucket
resize -> bigger table
treeify -> heavy bucket

17. Interview questions to revise

Why does HashMap use hashCode() and equals()? What is a bucket? Why are custom objects tricky as keys? What is load factor? What is threshold? Why does HashMap resize? When does treeification happen? How is HashSet related to HashMap? Why is HashMap average $O(1)$?

18. Final revision sheet

Topic	Simple meaning
HashMap	Key-value storage with hashing
Bucket	Array index where nodes are stored
hashCode()	Finds bucket
equals()	Finds exact key
Collision	Different keys in same bucket
Resize	Create bigger array and rehash nodes
Threshold	Size limit before resize
Load factor	How full the map can get
Treeification	Linked list bucket becomes tree
HashSet	HashMap wrapper for unique elements
TreeMap	Sorted key-value map using tree

Interview rule: first explain the idea in simple words, then explain the internal flow, then mention the complexity. That order usually sounds strongest in interviews.